

IMPLEMENTASI ALGORITMA GREEDY DALAM PEMECAHAN BOT PERMAINAN TANK ROYALE TUGAS BESAR



Oleh Kelompok : 10 (BSKGARED)

Stepan Immanuel Simbolon	124140130
Gede Valendra	124140142
Faisal H. Sinambela	124140040

Dosen Pengampu: Winda Yulita, M.Cs.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
LAMPUNG SELATAN**

2026

DAFTAR ISI

DAFTAR ISI.....	2
BAB 1	4
PENDAHULUAN	4
1. Deskripsi Tugas	4
BAB II.....	9
LANDASAN TEORI.....	9
2.1 Dasar Teori Algoritma Greedy Secara Umum.....	9
2.2 Cara Kerja Program Secara Umum.....	9
2.2.1 Mekanisme Kerja Bot pada Robocode	9
2.2.2 Implementasi Algoritma Greedy pada Bot	10
2.2.3 Prosedur Menjalankan Bot.....	10
BAB III	11
APLIKASI STRATEGI GREEDY	11
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy	11
3.1.1 Himpunan Kandidat.....	11
3.1.2 Himpunan Solusi.....	11
3.1.3 Fungsi Seleksi	11
3.1.4 Fungsi Solusi.....	11
3.1.5 Fungsi Kelayakan.....	11
3.1.6 Fungsi Objektif	12
3.2 Eksplorasi 4 Alternatif Solusi Greedy	12
3.2.1 Alternatif 1: BigBoss — Greedy Ideal Distance.....	12
3.2.2 Alternatif 2: LockBot — Greedy Candidate Position Scoring	12
3.2.3 Alternatif 3: GreedyDuelist — Greedy Hybrid Target Scoring.....	13
3.2.4 Alternatif 4: MinimumDangerBot-Greedy Minimum Danger	13
3.3 Analisis Efisiensi dan Efektivitas Alternatif Greedy	14
3.4 Strategi Greedy yang Dipilih	14
BAB IV	16
IMPLEMENTASI DAN PENGUJIAN	16
4.1 Implementasi 4 Alternatif Solusi	16
4.1.1 Pseudocode Bot EvasiveCircleBot	16
4.1.2 Pseudocode Bot RamFire.....	18

4.1.3 Pseudocode Bot Justinian	20
4.1.4 Pseudocode BSKGARED (Utama).....	26
4.2 Struktur Data, Fungsi, dan Prosedur pada Bot Utama	31
4.3 Pengujian Bot.....	32
4.4 Analisis Hasil Pengujian	33
BAB V	35
KESIMPULAN DAN SARAN.....	35
5.1 Kesimpulan	35
5.2 Saran	35
LAMPIRAN.....	36
Lampiran 1. Tautan Repository GitHub	36
Lampiran 2. Video Demo	36
DAFTAR PUSTAKA	37

BAB 1

PENDAHULUAN

1. Deskripsi Tugas

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.

- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih

ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

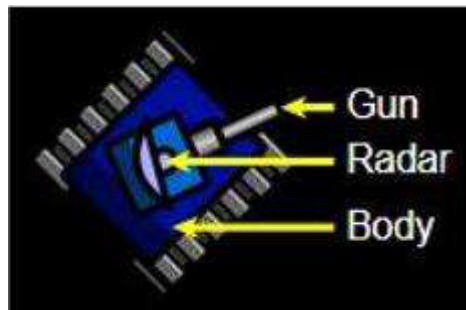
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

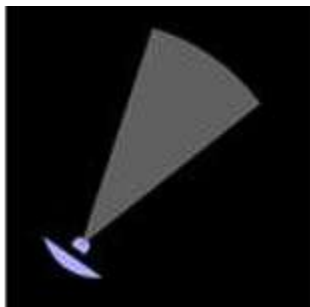
Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar. Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan poin sebesar *damage* yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan poin sebesar 20% dari *damage* yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan 50 poin.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan poin sebesar 2 kalinya *damage* yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan poin sebesar 30% dari *damage* yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

BAB II

LANDASAN TEORI

2.1 Dasar Teori Algoritma Greedy Secara Umum

Algoritma greedy merupakan algoritma yang bekerja dengan memilih keputusan terbaik pada setiap langkah berdasarkan kondisi saat itu. Kata greedy memiliki arti rakus atau tamak, karena algoritma ini selalu mengambil pilihan yang dianggap paling menguntungkan tanpa mengevaluasi seluruh kemungkinan solusi di masa depan.

Dalam algoritma greedy, keputusan yang diambil bersifat lokal. Artinya, sistem memilih solusi terbaik pada satu kondisi tertentu dengan harapan keputusan tersebut dapat menghasilkan solusi global yang baik. Meskipun tidak selalu menghasilkan solusi paling optimal, algoritma greedy banyak digunakan karena prosesnya cepat, sederhana, dan cocok untuk permasalahan real-time.

Pada permainan Robocode Tank Royale, algoritma greedy cocok digunakan karena bot harus mengambil keputusan dalam waktu singkat pada setiap tick permainan. Bot harus menentukan arah movement, target musuh, arah radar, kekuatan peluru, dan strategi menghindari serangan secara cepat. Oleh karena itu, greedy dapat digunakan untuk memilih tindakan terbaik berdasarkan informasi yang tersedia saat itu.

2.2 Cara Kerja Program Secara Umum

2.2.1 Mekanisme Kerja Bot pada Robocode

Secara umum, bot bekerja secara berulang selama pertandingan berlangsung. Pada setiap tick, bot membaca kondisi arena melalui radar. Ketika musuh terdeteksi, bot menyimpan informasi seperti posisi musuh, jarak musuh, energi musuh, kecepatan musuh, dan arah gerak musuh.

Setelah informasi diperoleh, bot menggunakan strategi greedy untuk menentukan aksi terbaik. Aksi tersebut dapat berupa memilih target, bergerak menuju posisi tertentu, menghindari dinding, mengunci radar ke arah musuh, memprediksi posisi musuh, dan menentukan kekuatan peluru.

Pada *GreedyDuelist*, bot menyimpan data musuh dalam struktur dictionary, kemudian memilih target terbaik menggunakan nilai skor berdasarkan jarak, energi musuh, kesegaran data scan, dan stabilitas target. Bot juga membedakan movement antara mode 1 vs 1 dan mode multi-bot. Jika hanya satu musuh, bot menggunakan duel movement. Jika terdapat lebih dari satu musuh, bot menggunakan multiwave movement agar lebih aman dari banyak arah serangan.

2.2.2 Implementasi Algoritma Greedy pada Bot

Algoritma greedy diimplementasikan dengan cara memberikan nilai pada setiap pilihan aksi. Setiap aksi memiliki keuntungan dan risiko. Aksi dengan nilai terbaik akan dipilih sebagai keputusan pada tick tersebut.

Sebagai contoh, dalam pemilihan target, bot dapat memberikan skor lebih tinggi kepada musuh yang jaraknya dekat, energinya rendah, dan baru saja terdeteksi radar. Dalam movement, bot dapat memilih posisi yang jaraknya ideal dari musuh, tidak dekat dinding, dan memberikan peluang lebih besar untuk menghindari peluru. Dalam shooting, bot memilih fire power berdasarkan jarak musuh dan energi bot.

Dengan cara tersebut, bot tidak memilih aksi secara acak, tetapi memilih aksi yang paling menguntungkan berdasarkan kondisi saat itu.

2.2.3 Prosedur Menjalankan Bot

Secara umum, bot dijalankan melalui game engine Robocode Tank Royale. File bot dibuat menggunakan bahasa C# dan disertai file konfigurasi bot dalam format JSON. Setelah bot dibuat, bot dimasukkan ke dalam folder bot yang sesuai, kemudian game engine dijalankan. Bot dapat diuji dengan melawan bot lain dalam mode 1 vs 1 atau mode multi-bot.

Langkah umum dalam menjalankan bot sebagai berikut:

1. Menyiapkan file kode bot dalam bahasa C#.
2. Menyiapkan file konfigurasi bot, misalnya *greedyduelist.json*.
3. Menjalankan game engine Robocode Tank Royale.
4. Memilih bot yang akan diuji.
5. Menjalankan pertandingan.
6. Mencatat hasil skor, damage, survival, dan peringkat akhir.

BAB III

APLIKASI STRATEGI GREEDY

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen Greedy

Dalam algoritma greedy terdapat beberapa elemen utama, yaitu:

3.1.1 Himpunan Kandidat

Himpunan kandidat adalah kumpulan pilihan yang dapat dipilih oleh bot. Dalam Robocode Tank Royale, kandidat dapat berupa target musuh, arah movement, posisi tujuan, kekuatan tembakan, dan arah radar.

3.1.2 Himpunan Solusi

Himpunan solusi adalah keputusan yang telah dipilih oleh bot dari himpunan kandidat. Contohnya adalah target utama yang dipilih, posisi tujuan yang akan ditempuh, dan *fire power* yang digunakan.

3.1.3 Fungsi Seleksi

Fungsi seleksi digunakan untuk memilih kandidat terbaik berdasarkan heuristic tertentu. Contohnya adalah memilih musuh dengan jarak terdekat, memilih posisi dengan risiko paling kecil, atau memilih target dengan energi paling rendah

3.1.4 Fungsi Solusi

Kriteria yang menyatakan bahwa sekumpulan aksi yang dipilih telah berhasil dieksekusi secara lengkap dan valid oleh robot untuk merespons kondisi arena pada *tick* tersebut.

3.1.5 Fungsi Kelayakan

Himpunan Fungsi kelayakan digunakan untuk memeriksa apakah kandidat dapat digunakan atau tidak. Contohnya adalah memastikan posisi tujuan tidak berada di luar arena, tidak terlalu dekat dengan dinding, dan tidak membuat bot bergerak ke posisi berbahaya.

3.1.6 Fungsi Objektif

Fungsi objektif adalah tujuan utama yang ingin dicapai. Pada permainan Robocode Tank Royale, fungsi objektifnya adalah memperoleh skor setinggi mungkin dengan cara meningkatkan damage, menghindari peluru, bertahan hidup lebih lama, dan mengalahkan musuh.

3.2 Eksplorasi 4 Alternatif Solusi Greedy

3.2.1 Alternatif 1: BigBoss — Greedy Ideal Distance

BigBoss menggunakan strategi greedy berdasarkan jarak ideal terhadap musuh. Bot akan mendekat jika musuh terlalu jauh, menjauh jika musuh terlalu dekat, dan melakukan strafe jika berada pada jarak ideal. Strategi ini bertujuan menjaga bot tetap berada pada posisi tembak yang efektif.

Heuristic yang digunakan adalah:

- jika jarak musuh lebih besar dari batas maksimum, bot mendekat;
- jika jarak musuh lebih kecil dari batas minimum, bot menjauh;
- jika jarak berada pada area ideal, bot melakukan strafe;
- jika mendeteksi musuh menembak melalui penurunan energi, bot melakukan dodge.

Kode BigBoss menunjukkan penggunaan MIN_DISTANCE, IDEAL_DISTANCE, dan MAX_DISTANCE untuk menentukan apakah bot harus mendekat, menjauh, atau melakukan strafe.

3.2.2 Alternatif 2: LockBot — Greedy Candidate Position Scoring

LockBot menggunakan strategi greedy dengan mengevaluasi beberapa kandidat posisi. Bot membuat beberapa kemungkinan arah dan jarak gerak, kemudian menghitung skor dari setiap kandidat. Posisi dengan skor tertinggi dipilih sebagai posisi tujuan.

Heuristic yang digunakan adalah:

- Menjaga jarak ideal dari musuh;
- Memaksimalkan gerakan lateral agar sulit terkena peluru;
- Menghindari posisi dekat dinding;
- Menghindari pojok arena;
- Menghindari duel dekat ketika energi rendah.

LockBot memiliki fungsi `ChooseGreedyMove()` dan `ScorePosition()` untuk mengevaluasi kandidat posisi. Fungsi tersebut memberikan penalti pada posisi berbahaya dan bonus pada posisi yang mendukung gerakan lateral serta jarak ideal.

3.2.3 Alternatif 3: GreedyDuelist — Greedy Hybrid Target Scoring

GreedyDuelist menggunakan strategi greedy hybrid. Bot tidak hanya memilih movement, tetapi juga memilih target terbaik berdasarkan skor. Target yang dipilih adalah musuh dengan kombinasi terbaik dari jarak, energi, kesegaran data scan, dan stabilitas target.

Heuristic yang digunakan adalah:

- memilih musuh yang lebih dekat;
- memilih musuh dengan energi lebih rendah;
- memilih musuh dengan data scan terbaru;
- mempertahankan target agar tidak sering berganti;
- mengejar posisi terakhir musuh jika target hilang;
- membedakan strategi movement antara mode duel dan multi-bot.

GreedyDuelist memiliki fungsi `SelectBestTarget()` yang menghitung skor target menggunakan `distanceScore`, `weakScore`, dan `freshScore`. Bot juga memberikan bonus pada target yang sedang dikunci agar target tidak terlalu sering berganti.

3.2.4 Alternatif 4: MinimumDangerBot-Greedy Minimum Danger

MinimumDangerBot menggunakan strategi greedy berdasarkan nilai bahaya posisi. Bot mengevaluasi beberapa kandidat posisi di sekitar bot, kemudian memilih posisi dengan nilai bahaya paling kecil. Strategi ini lebih berfokus pada survival dibanding serangan langsung.

Heuristic yang digunakan adalah:

- memilih posisi dengan nilai bahaya paling rendah;
- menjauhi musuh yang berbahaya;
- menghindari lintasan peluru virtual;
- menghindari dinding dan pojok arena;

- menggunakan data gerakan musuh untuk prediksi.

Kode Woff/MinimumDangerBot menunjukkan penggunaan daftar peluru virtual, data musuh, dan fungsi CalcGrav() untuk menghitung nilai bahaya pada kandidat posisi.

Bot memilih posisi dengan nilai gravitasi paling kecil sebagai tujuan movement.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Greedy

Alternatif	Efisiensi	Efektivitas	Kelebihan	Kekurangan
BigBoss	Tinggi	Cukup baik untuk 1 vs 1	Sederhana dan cepat	Kurang kuat untuk multi-bot
LockBot	Sedang	Baik untuk duel	Movement lebih terarah	Kandidat posisi masih terbatas
GreedyDuelist	Sedang	Baik untuk 1 vs 1 dan multi-bot	Seimbang antara serangan dan bertahan	Parameter lebih kompleks
MinimumDangerBot	Lebih berat	Baik untuk survival	Mampu memilih posisi aman	Perhitungan lebih banyak

BigBoss memiliki efisiensi tinggi karena hanya mengambil keputusan berdasarkan jarak. Namun, strategi ini kurang efektif ketika menghadapi banyak musuh. LockBot lebih efektif karena menggunakan scoring posisi, tetapi masih terbatas pada kandidat posisi tertentu. GreedyDuelist lebih seimbang karena menggabungkan target scoring, radar lock, movement adaptif, dan pengaturan energi. MinimumDangerBot memiliki survival tinggi, tetapi lebih berat karena harus menghitung nilai bahaya dari banyak kandidat posisi.

3.4 Strategi Greedy yang Dipilih

Strategi greedy yang dipilih sebagai bot utama adalah **GreedyDuelist**. Bot ini dipilih karena memiliki kombinasi strategi yang paling seimbang dibandingkan alternatif lain. GreedyDuelist tidak hanya fokus menyerang, tetapi juga mempertimbangkan stabilitas target, efisiensi energi, radar lock, movement adaptif, dan kondisi multi-bot.

Alasan pemilihan GreedyDuelist adalah sebagai berikut:

1. Dapat digunakan pada pertandingan 1 vs 1 maupun multi-bot.

2. Memiliki sistem target scoring yang lebih stabil.
3. Tidak mudah kehilangan musuh pada jarak jauh.
4. Mampu mengejar posisi terakhir target ketika radar kehilangan musuh.
5. Mengatur fire power agar energi tidak cepat habis.
6. Memiliki movement berbeda untuk duel dan multi-bot.

Dengan pertimbangan tersebut, GreedyDuelist dipilih sebagai bot utama yang akan dikumpulkan dan digunakan dalam kompetisi.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi 4 Alternatif Solusi

4.1.1 Pseudocode Bot EvasiveCircleBot

```
ALGORITMA EvasiveCircleBot

DEKLARASI VARIABEL:
    moveDir : integer // Mengatur arah gerak (1 untuk maju, -1 untuk mundur)
    turnDir : integer // Mengatur arah putaran (1 untuk kanan, -1 untuk kiri)

PROSEDUR Utama()
    Inisialisasi bot menggunakan berkas konfigurasi "alt.json"
    Mulai jalankan bot
END PROSEDUR

PROSEDUR Run()
    // 1. Inisialisasi Kosmetik dan Kecepatan Bot
    Atur Warna Body = DarkBlue
    Atur Warna Turret = Cyan
    Atur Warna Radar = Yellow
    Atur Warna Peluru = White
    Atur MaxSpeed = 7

    // 2. Mengaktifkan Radar Berputar Terus-menerus (Mencari Musuh)
    SetTurnRadarRight(Infinity)

    // 3. Loop Utama Pergerakan Bot (Pola Lingkaran / Spiral)
    SELAMA bot masih berjalan (IsRunning) NYATAKAN:
        SetTurnRight(35 * turnDir) // Mengatur derajat putaran roda
        SetForward(250 * moveDir) // Mengatur jarak gerak maju/mundur
        SetTurnGunRight(25) // Sedikit memutar meriam secara berkala
        Go() // Eksekusi semua perintah di atas secara
simultan
    END SELAMA
END PROSEDUR

PROSEDUR OnScannedBot(Event e)
    // Abaikan jika bot yang terdeteksi adalah teman satu tim
    JIKA IsTeammate(e.ScannedBotId) MAKA
        KELUAR PROSEDUR
    END JIKA
```



```

// Hitung jarak dan sudut relatif meriam terhadap musuh
distance = JarakKe(e.X, e.Y)
gunBearing = SudutMeriamKe(e.X, e.Y)

// Arahkan meriam secara presisi ke koordinat musuh
SetTurnGunLeft(gunBearing)

// Logika Penembakan Efisiensi Energi (Greedy Fire Power)
JIKA (Mutlak(gunBearing) < 8) DAN (GunHeat == 0) MAKA
    JIKA (distance < 120) DAN (Energy > 30) MAKA
        Fire(3) // Tembak kekuatan penuh jika sangat dekat dan energi
aman
    ATAU JIKA (distance < 350) MAKA
        Fire(2) // Tembak kekuatan sedang pada jarak menengah
    LAINNYA
        Fire(1) // Tembak kekuatan hemat untuk jarak jauh
    END JIKA
END JIKA

// Logika Menghindar (Evasive) jika musuh terlalu dekat
JIKA distance < 180 MAKA
    moveDir = moveDir * -1 // Balikkan arah gerak utama
    SetBack(180)           // Mundur sejauh 180 unit
END JIKA

Rescan() // Paksa radar untuk langsung memindai ulang
END PROSEDUR

PROSEDUR OnHitByBullet(Event e)
    // Taktik menghindar darurat saat terkena tembakan peluru musuh
    moveDir = moveDir * -1 // Balikkan arah gerak
    turnDir = turnDir * -1 // Balikkan arah putaran lingkaran

    SetTurnRight(70 * turnDir) // Putar badan robot dengan sudut tajam
    SetForward(220 * moveDir)  // Kabur menjauh dari posisi semula
END PROSEDUR

PROSEDUR OnHitWall(Event e)
    // Penanganan jika robot menabrak dinding arena
    moveDir = moveDir * -1 // Balikkan arah gerak
    SetBack(180)           // Mundur menjauhi dinding
END PROSEDUR

PROSEDUR OnHitBot(Event e)
    // Penanganan jika terjadi tabrakan antar robot (Ramming)
    JIKA e.IsRammed MAKA
        moveDir = moveDir * -1 // Balikkan arah gerak
        SetBack(120)           // Mundur darurat
    END JIKA
END PROSEDUR

```

```

END JIKA

// Balas tembak musuh yang menabrak dengan kekuatan penuh
gunBearing = SudutMeriamKe(e.X, e.Y)
TurnGunLeft(gunBearing)    // Kunci meriam langsung ke lawan

JIKA GunHeat == 0 MAKA
    Fire(3)                // Tembak langsung dengan daya rusak maksimal
END JIKA
END PROSEDUR

```

4.1.2 Pseudocode Bot RamFire

```

ALGORITMA RamFire

DEKLARASI KONSTANTA DAN VARIABEL:
    turnDirection : integer = 1    // Arah putaran bot (1 untuk kanan, -1 untuk
    kiri)
    SAFE_DISTANCE : float = 300    // Batas jarak ideal aman dari musuh
    DISTANCE_TOLERANCE : float = 20 // Toleransi jarak agar tidak konstan
    maju-mundur

PROSEDUR Utama()
    Inisialisasi bot menggunakan berkas konfigurasi "alt.json"
    Mulai jalankan bot
END PROSEDUR

PROSEDUR Run()
    // 1. Pengaturan Kosmetik Bot (Warna Abu-abu & Kuning)
    Atur Warna Body = RGB(0x99, 0x99, 0x99)
    Atur Warna Turret = RGB(0x88, 0x88, 0x88)
    Atur Warna Radar = RGB(0x66, 0x66, 0x66)
    Atur Warna Scan = Kuning

    // 2. Mengaktifkan Fitur Independensi Perputaran Komponen
    AdjustGunForBodyTurn = Benar
    AdjustRadarForBodyTurn = Benar
    AdjustRadarForGunTurn = Benar

    // 3. Mengatur Batas Maksimum Kecepatan Robot
    MaxSpeed = 8

    // 4. Loop Utama untuk Pemindaian Lingkungan
    SELAMA bot masih berjalan (IsRunning) NYATAKAN:
        TurnRadarRight(360)    // Memutar radar penuh 360 derajat untuk
        mendeteksi musuh
    END SELAMA
END PROSEDUR

```

```

PROSEDUR OnScannedBot(Event e)
    // Hitung jarak matematis dan sudut meriam terhadap koordinat target
    distance = JarakKe(e.X, e.Y)
    gunBearing = SudutMeriamKe(e.X, e.Y)

    // Hadapkan meriam secara linear ke arah target
    TurnGunLeft(gunBearing)

    // Logika Greedy Penembakan Target (Sarat Tembak Maksimal)
    JIKA (GunHeat == 0) DAN (Abs(SudutMeriamKe(e.X, e.Y)) < 10) MAKA
        Fire(3) // Eksekusi tembakan dengan daya hancur tertinggi (Power 3)
    END JIKA

    // Logika Pergerakan Berdasarkan Jarak Aman (Greedy Range Adjustment)
    JIKA distance < (SAFE_DISTANCE - DISTANCE_TOLERANCE) MAKA
        // Kondisi Musuh Terlalu Dekat: Robot menjauh demi keamanan
        TurnRight(25 * turnDirection)
        Back(120)
    ATAU JIKA distance > (SAFE_DISTANCE + DISTANCE_TOLERANCE) MAKA
        // Kondisi Musuh Terlalu Jauh: Robot memotong sudut untuk mendekat
        TurnRight(20 * turnDirection)
        Forward(80)
    LAINNYA
        // Kondisi Jarak Sudah Ideal: Tetap bergerak kecil secara melingkar
        TurnRight(15 * turnDirection)
        Forward(50)
    END JIKA

    Rescan() // Melakukan pemindaian ulang instan
END PROSEDUR

PROSEDUR OnHitBot(Event e)
    // Sinkronisasi meriam ke koordinat musuh yang menabrak
    TurnGunLeft(SudutMeriamKe(e.X, e.Y))

    // Tembak langsung jika mekanisme meriam siap (tidak panas)
    JIKA GunHeat == 0 MAKA
        Fire(3)
    END JIKA

    // Taktik Pelepasan Diri Pasca Tabrakan
    Back(140) // Mundur darurat
    TurnRight(45 * turnDirection) // Memutar arah badan robot
    turnDirection = turnDirection * -1 // Balikkan orientasi putaran utama
robot

    Rescan()

```

```

END PROSEDUR

PROSEDUR OnHitWall(Event e)
    // Taktik Penanganan Benturan Dinding Arena
    Back(120) // Mundur menjauhi tembok
    TurnRight(90 * turnDirection) // Berputar tegak lurus untuk
    merubah jalur
    turnDirection = turnDirection * -1 // Balikkan orientasi putaran utama
    robot

    Rescan()
END PROSEDUR

FUNGSI Abs(value : float) -> float
    // Implementasi manual untuk mendapatkan nilai mutlak (absolut)
    JIKA value < 0 MAKA
        KEMBALIKAN -value
    LAINNYA
        KEMBALIKAN value
    END JIKA
END FUNGSI

```

4.1.3 Pseudocode Bot Justinian

```

ALGORITMA Justinian

DEKLARASI STRUKTUR DATA:
    STRUKTUR EnemyInfo:
        Id, LastSeen : integer
        X, Y, Energy, PreviousEnergy, Distance, Direction, Speed : float
        RecentlyFired : boolean

DEKLARASI KONSTANTA DAN VARIABEL GLOBAL:
    enemies : Dictionary dari <integer, EnemyInfo> // Penyimpanan data semua
    musuh
    target : EnemyInfo // Pointer ke target utama
    saat ini
    tick : integer = 0 // Pencatat siklus
    permainan (waktu)
    moveDirection : integer = 1 // Arah gerak (1 maju, -1
    mundur)
    random : ObjectRandom // Pembangkit nilai acak
    WALL_MARGIN : float = 95 // Batas aman jarak
    dinding arena
    MIN_DISTANCE : float = 135 // Batas jarak minimum bot
    dari musuh
    IDEAL_DISTANCE : float = 220 // Titik jarak paling
    ideal dari musuh

```

```

    MAX_DISTANCE : float = 420                                // Batas jarak maksimum
lingkaran orbit

PROSEDUR Utama()
    Inisialisasi bot menggunakan berkas konfigurasi "alt.json"
    Mulai jalankan bot
END PROSEDUR

PROSEDUR Run()
    // 1. Kustomisasi Kosmetik Visual Bot
    BodyColor = Hitam; TurretColor = Merah; RadarColor = Kuning
    BulletColor = Jingga; ScanColor = Abu-abu; TracksColor = Hijau; GunColor =
Putih
    MaxSpeed = 8

    // 2. Independensi Perputaran Roda, Meriam, dan Radar
    AdjustGunForBodyTurn = Benar
    AdjustRadarForBodyTurn = Benar
    AdjustRadarForGunTurn = Benar

    SetFireAssist(Benar)                                     // Mengaktifkan bantuan otomatisasi
internal
    SetTurnRadarRight(Infinity)                             // Putar radar terus-menerus di awal
laga

    // 3. Loop Siklus Utama Pertandingan
    SELAMA bot masih berjalan (IsRunning) NYATAKAN:
        tick = tick + 1

        RemoveOldEnemies()                                  // Buang data musuh yang sudah
kadaluarsa (hilang)
        SelectBestTargetGreedy()                            // Evaluasi ulang musuh terbaik
menggunakan Greedy Scoring
        DoRadar()                                           // Eksekusi pergerakan radar penyekanan
target
        DoMovement()                                       // Eksekusi pergerakan manuver
menghindar/orbiting

        JIKA target != Kosong MAKA
            AimAndFire(target)                             // Arahkan meriam dengan prediksi linear
dan tembak
        END JIKA

        Go()                                               // Kirim dan jalankan semua aksi ke
server secara simultan
    END SELAMA
END PROSEDUR

```

```

PROSEDUR OnScannedBot(Event e)
    Abaikan JIKA IsTeammate(e.ScannedBotId)

    distance = JarakKe(e.X, e.Y)
    previousEnergy = e.Energy
    recentlyFired = Salah

    // Logika Deteksi Tembakan Musuh Berdasarkan Penurunan Energi Spontan
    JIKA enemies mengandung kunci(e.ScannedBotId) MAKA
        previousEnergy = enemies[e.ScannedBotId].Energy
        energyDrop = previousEnergy - e.Energy
        // Jika energi musuh berkurang di antara rentang peluru valid
        recentlyFired = (energyDrop > 0.09) DAN (energyDrop <= 3.01)
    END JIKA

    // Perbarui database informasi musuh
    enemies[e.ScannedBotId] = EnemyInfo baru dengan data {
        Id = e.ScannedBotId, X = e.X, Y = e.Y, Energy = e.Energy,
        PreviousEnergy = previousEnergy, Distance = distance,
        Direction = e.Direction, Speed = e.Speed, LastSeen = tick,
        RecentlyFired = recentlyFired
    }

    SelectBestTargetGreedy()           // Hitung ulang bobot target

    // Taktik Tanggap Darurat Tembakan Musuh (Dodge Mechanism)
    JIKA (target != Kosong) DAN (target.Id == e.ScannedBotId) MAKA
        JIKA recentlyFired DAN (distance < 450) MAKA
            moveDirection = moveDirection * -1 // Balikkan arah gerak mendadak
        (Dodge)
        END JIKA
        AimAndFire(target)
    END JIKA
END PROSEDUR

PROSEDUR SelectBestTargetGreedy()
    bestScore = -Infinity
    bestEnemy = Kosong

    UNTUK SETIAP enemy DI DALAM enemies.Values LAKUKAN:
        // Abaikan musuh yang sudah tidak terlihat lebih dari 55 tick
        JIKA tick - enemy.LastSeen > 55 MAKA LAKUKAN LANJUTKAN_LOOP

        // KANDIDAT SKOR GREEDY (HEURISTIC)
        distanceScore = 1600 / Maksimum(enemy.Distance, 1)
        lowEnergyScore = 900 / Maksimum(enemy.Energy, 1)

        killBonus = (enemy.Energy < 18) ? 450 : 0

```

```

        veryLowBonus = (enemy.Energy < 8) ? 500 : 0
        closeCombatBonus = (enemy.Distance < 260) ? 220 : 0
        tooFarPenalty = (enemy.Distance > 520) ? -300 : 0
        wallBonus = IsNearWall(enemy.X, enemy.Y) ? 160 : 0
        easyHitBonus = Maksimum(0, 180 - Mutlak(enemy.Speed) * 25)
        energySafetyPenalty = (Energy < 20 DAN enemy.Distance > 300) ? -350 :
0

        // Total akumulasi nilai kelayakan kandidat target
        score = distanceScore + lowEnergyScore + killBonus + veryLowBonus +
            closeCombatBonus + wallBonus + easyHitBonus + tooFarPenalty +
energySafetyPenalty

        // Fungsi Seleksi: Ambil skor tertinggi
        JIKA score > bestScore MAKA
            bestScore = score
            bestEnemy = enemy
        END JIKA
    END UNTUK

    target = bestEnemy // Himpunan Solusi Target Terpilih
END PROSEDUR

PROSEDUR AimAndFire(EnemyInfo enemy)
    power = GetEfficientFirePower(enemy) // Ambil fire power berbasis kondisi
energi lokal

    JIKA (power <= 0) ATAU (GunHeat > 0) MAKA KELUAR PROSEDUR

    // ALGORITMA PREDIKSI LINEAR TARGETING
    bulletSpeed = 20 - 3 * power
    time = enemy.Distance / bulletSpeed

    predictedX = enemy.X + Sin(ToRad(enemy.Direction)) * enemy.Speed * time
    predictedY = enemy.Y + Cos(ToRad(enemy.Direction)) * enemy.Speed * time

    // Batasi prediksi koordinat agar tidak menembak ke luar dinding arena
    predictedX = Clamp(predictedX, WALL_MARGIN, ArenaWidth - WALL_MARGIN)
    predictedY = Clamp(predictedY, WALL_MARGIN, ArenaHeight - WALL_MARGIN)

    gunTurn = SudutMeriamKe(predictedX, predictedY)
    SetTurnGunLeft(gunTurn) // Putar meriam ke titik prediksi masa
depan musuh

    // Toleransi Akurasi Bidikan Berdasarkan Jarak
    JIKA enemy.Distance < 140 MAKA aimTolerance = 10
    ATAU JIKA enemy.Distance < 280 MAKA aimTolerance = 6
    LAINNYA aimTolerance = 3.5

```

```

    END JIKA

    // Eksekusi tembakan jika sudut meriam sudah presisi di dalam batas
    toleransi
    JIKA Mutlak(gunTurn) <= aimTolerance MAKA
        SetFire(power)
    END JIKA
END PROSEDUR

FUNGSI GetEfficientFirePower(EnemyInfo enemy) -> float
    // Fungsi Manajemen Energi Hemat (Greedy Fire Power Allocation)
    JIKA Energy < 8 MAKA KEMBALIKAN 0
    JIKA Energy < 16 MAKA KEMBALIKAN (enemy.Distance < 180) ? 0.9 : 0.6
    JIKA enemy.Energy < 5 MAKA KEMBALIKAN 0.8
    JIKA (enemy.Energy < 12) DAN (enemy.Distance < 260) MAKA KEMBALIKAN 1.4

    // Alokasi kekuatan berdasarkan jarak tembak aman
    JIKA enemy.Distance < 90 MAKA KEMBALIKAN (Energy > 25) ? 3.0 : 1.8
    JIKA enemy.Distance < 170 MAKA KEMBALIKAN (Energy > 22) ? 2.5 : 1.5
    JIKA enemy.Distance < 320 MAKA KEMBALIKAN (Energy > 18) ? 1.8 : 1.0
    JIKA enemy.Distance < 520 MAKA KEMBALIKAN (Energy > 35) ? 1.2 : 0.7

    KEMBALIKAN 0.5
END FUNGSI

PROSEDUR DoMovement()
    // Kriteria Kelayakan Utama: Deteksi kedekatan dinding untuk menghindari
    JIKA NearWallSelf() MAKA
        EscapeWall()
        KELUAR PROSEDUR
    END JIKA

    // Pergerakan acak jelajah jika target belum ditemukan
    JIKA target == Kosong MAKA
        SetTurnRight(25)
        SetForward(140 * moveDirection)
        JIKA tick MODULO 45 == 0 MAKA moveDirection = moveDirection * -1
        KELUAR PROSEDUR
    END JIKA

    distance = JarakKe(target.X, target.Y)
    bearing = SudutBadanKe(target.X, target.Y)

    // Strategi Gerak 1: Kejar (Chase) jika musuh terlalu jauh
    JIKA distance > MAX_DISTANCE MAKA
        chaseAngle = bearing + 18 * moveDirection
        SetTurnLeft(chaseAngle)
        SetForward(Min(distance - IDEAL_DISTANCE, 260))

```



```

        KELUAR PROSEDUR
    END JIKA

    // Strategi Gerak 2: Mundur (Retreat) jika musuh terlalu dekat menjepit
    JIKA distance < MIN_DISTANCE MAKA
        retreatAngle = bearing + 70 * moveDirection
        SetTurnLeft(retreatAngle)
        SetBack(135)
        KELUAR PROSEDUR
    END JIKA

    // Strategi Gerak 3: Mengorbit Melingkar (Orbiting & Wobbling) pada Jarak Ideal
    orbitOffset = 90
    JIKA distance < IDEAL_DISTANCE MAKA orbitOffset = 105
    ATAU JIKA distance > IDEAL_DISTANCE + 70 MAKA orbitOffset = 65
    END JIKA

    randomWobble = (tick MODULO 20 < 10) ? 12 : -12
    turn = bearing + (orbitOffset * moveDirection) + randomWobble
    SetTurnLeft(turn)

    // Pembalikan arah gerak berkala secara probabilistik (Anti-Prediction Movement)
    JIKA (target.RecentlyFired DAN target.Distance < 420) ATAU (tick MODULO 32 == 0) ATAU (NilaiAcak()) < 0.025) MAKA
        moveDirection = moveDirection * -1
    END JIKA

    moveAmount = 155
    JIKA distance > IDEAL_DISTANCE + 80 MAKA moveAmount = 190
    ATAU JIKA distance < IDEAL_DISTANCE - 60 MAKA moveAmount = 120
    END JIKA

    SetForward(moveAmount * moveDirection)
END PROSEDUR

PROSEDUR DoRadar()
    JIKA target == Kosong MAKA
        SetTurnRadarRight(Infinity)
    LAINNYA
        // Kunci Radar Sempit (Radar Narrow Lock Mechanism)
        radarTurn = SudutRadarKe(target.X, target.Y)
        SetTurnRadarLeft(radarTurn * 2.2)
    END JIKA
END PROSEDUR

PROSEDUR OnHitByBullet(Event e)

```

```

    moveDirection = moveDirection * -1
    // Hitung sudut melarikan diri tegak lurus dari arah kedatangan peluru
    escapeAngle = NormalizeRelativeAngle((e.Bullet.Direction + 90) -
Direction)
    SetTurnLeft(escapeAngle)
    SetForward(180 * moveDirection)
END PROSEDUR

PROSEDUR OnHitBot(Event e)
    Abaikan JIKA IsTeammate(e.VictimId)

    gunTurn = SudutMeriamKe(e.X, e.Y)
    SetTurnGunLeft(gunTurn)

    JIKA GunHeat == 0 MAKA
        SetFire((Energy > 25) ? 3 : 1.2)
    END JIKA

    moveDirection = moveDirection * -1
    SetBack(100)
    SetTurnRight(55)
END PROSEDUR

PROSEDUR OnHitWall(Event e)
    EscapeWall()
END PROSEDUR

PROSEDUR OnBulletHit(Event e)
    // Jika peluru kita berhasil mengenai musuh, kunci target tersebut secara
pak

```

4.1.4 Pseudocode BSKGARED (Utama)

```

ALGORITMA BSKGARED_Advanced

DEKLARASI VARIABEL GLOBAL DAN KONSTANTA:
    moveDirection : integer = 1      // Mengatur arah gerak maju (1) atau
mundur (-1)
    strafeDirection : integer = 1    // Mengatur arah belok menyamping untuk
manuver menghindari
    lastEnemyEnergy : float = 100    // Menyimpan catatan energi terakhir
musuh
    WALL_MARGIN : float = 90         // Batas aman minimum terhadap posisi
dinding arena
    DANGER_MARGIN : float = 130      // Batas ambang bahaya untuk
memprediksi benturan dinding

PROSEDUR Utama()

```

```

    Inisialisasi bot menggunakan berkas konfigurasi "main.json"
    Mulai jalankan bot
END PROSEDUR

PROSEDUR Run()
    // 1. Kustomisasi Estetika Visual Robot
    BodyColor = DarkGoldenrod; TurretColor = Abu-abu; RadarColor =
DarkTurquoise
    BulletColor = Merah; ScanColor = DarkRed
    MaxSpeed = 8

    // 2. Mengaktifkan Independensi Putaran Komponen
    AdjustGunForBodyTurn = Benar
    AdjustRadarForBodyTurn = Benar

    // 3. Loop Siklus Utama Pertandingan
    SELAMA bot masih berjalan (IsRunning) NYATAKAN:
        SetTurnRadarRight(360)          // Putar radar penuh untuk mendeteksi
keberadaan musuh
        AvoidWallRoute()                // Navigasi taktis untuk menghindari
dinding arena
        Go()                            // Eksekusi seluruh rangkaian perintah
secara bersamaan
    END SELAMA
END PROSEDUR

PROSEDUR OnScannedBot(Event e)
    // Hitung parameter jarak dan sudut absolut posisi target
    distance = JarakKe(e.X, e.Y)
    absoluteBearing = SudutAbsolutKe(e.X, e.Y)

    // Kunci Radar Dinamis (Radar Lock Mechanism)
    radarTurn = NormalizeRelativeAngle(absoluteBearing - RadarDirection)
    SetTurnRadarLeft(radarTurn * 2.5)

    // LOGIKA GREEDY MENCHINDAR (Energy Drop Tembakan Peluru Lawan)
    energyDrop = lastEnemyEnergy - e.Energy
    JIKA (energyDrop > 0.09) DAN (energyDrop <= 3.01) MAKA
        moveDirection = moveDirection * -1      // Balikkan arah gerak secara
instan
        strafeDirection = strafeDirection * -1  // Ubah orientasi elakan
samping
        SetForward(190 * moveDirection)
    END JIKA

    lastEnemyEnergy = e.Energy                // Perbarui riwayat energi musuh

    // LOGIKA PERGERAKAN MANUEVER ORBITING (Greedy Positioning)

```

```

    JIKA IsNearWall() ATAU WillHitWall(170) MAKA
        MoveToSafeRoute() // Ambil jalur penyelamatan ke tengah
arena
    LAINNYA
        // Kalkulasi sudut gerak mengitari target (Orbiting Angle)
        orbitAngle = 90 - (22 * moveDirection)
        bodyTurn = NormalizeRelativeAngle(absoluteBearing + orbitAngle -
Direction)
        SetTurnLeft(bodyTurn)

        // Penyesuaian Jarak Ideal Berbasis Kondisi Lokal Jarak Target
        JIKA distance > 300 MAKA
            SetForward(210 * moveDirection) // Mendekat jika terlalu jauh
        ATAU JIKA distance < 170 MAKA
            SetBack(150) // Mundur menjauh jika terlalu
rapat
        LAINNYA
            SetForward(150 * moveDirection) // Menjaga jarak konstan
        END JIKA
    END JIKA

    // EVALUASI GREEDY OFFSETS BIDIKAN (Aim Offsets)
    JIKA distance > 450 MAKA aimOffset = -2
    ATAU JIKA distance > 250 MAKA aimOffset = -1
    ATAU JIKA distance > 120 MAKA aimOffset = 0
    LAINNYA aimOffset = 0.4
    END JIKA

    // Sinkronisasi Perputaran Arah Meriam
    gunTurn = NormalizeRelativeAngle((absoluteBearing - GunDirection) +
aimOffset)
    SetTurnGunLeft(gunTurn)

    // Eksekusi Tembakan dengan Proteksi Energi Bot (Greedy Fire Power)
    JIKA (GunHeat == 0) DAN (Mutlak(gunTurn) < GetAimTolerance(distance)) MAKA
        power = GetFirePower(distance)
        // Amankan sisa energi bot agar tidak mati bunuh diri akibat menembak
        Fire(Min(power, Energy - 0.2))
    END JIKA

    Rescan()
END PROSEDUR

PROSEDUR AvoidWallRoute()
    // Pengecekan kelayakan jalur terhadap batas area dinding
    JIKA IsNearWall() ATAU WillHitWall(130) MAKA
        MoveToSafeRoute() // Alihkan haluan ke posisi aman
    LAINNYA

```

```

        SetTurnRight(25 * strafeDirection) // Gerakan meliuk normal (Strafing)
        SetForward(160 * moveDirection)
    END JIKA
END PROSEDUR

FUNGSI IsNearWall() -> boolean
    // Memeriksa apakah koordinat riil robot melanggar batas perimeter dinding
    KEMBALIKAN (X < WALL_MARGIN) ATAU (Y < WALL_MARGIN) ATAU
                (X > ArenaWidth - WALL_MARGIN) ATAU (Y > ArenaHeight -
WALL_MARGIN)
END FUNGSI

FUNGSI WillHitWall(distanceAhead : float) -> boolean
    // Transformasi derajat arah ke bentuk Radian
    rad = Direction * PI / 180.0

    // Prediksi posisi matematis masa depan (Look-Ahead Koordinat)
    nextX = X + Sin(rad) * distanceAhead * moveDirection
    nextY = Y + Cos(rad) * distanceAhead * moveDirection

    // Validasi apakah titik prediksi menabrak zona bahaya dinding
    KEMBALIKAN (nextX < DANGER_MARGIN) ATAU (nextY < DANGER_MARGIN) ATAU
                (nextX > ArenaWidth - DANGER_MARGIN) ATAU (nextY > ArenaHeight -
DANGER_MARGIN)
END FUNGSI

PROSEDUR MoveToSafeRoute()
    // Menghitung titik pusat absolut arena pertandingan
    centerX = ArenaWidth / 2.0
    centerY = ArenaHeight / 2.0

    safeDirection = DirectionTo(centerX, centerY)
    turn = NormalizeRelativeAngle(safeDirection - Direction)

    moveDirection = 1 // Inisialisasi arah awal maju

    // Optimalisasi Gerak: Jika sudut terlalu besar, mundur dinilai lebih
efisien
    JIKA Mutlak(turn) > 90 MAKA
        turn = NormalizeRelativeAngle(turn + 180)
        moveDirection = -1
    END JIKA

    strafeDirection = strafeDirection * -1 // Ubah pola putaran agar tidak
terbaca musuh
    SetTurnLeft(turn)
    SetForward(220 * moveDirection) // Reposisi menjauh dari dinding
END PROSEDUR

```

```

FUNGSI GetFirePower(distance : float) -> float
    // Alokasi Kekuatan Peluru Berdasarkan Efisiensi Energi dan Jarak
    JIKA Energy < 15 MAKA KEMBALIKAN 1.2
    JIKA distance < 90 MAKA KEMBALIKAN 3.0
    JIKA distance < 180 MAKA KEMBALIKAN 2.7
    JIKA distance < 350 MAKA KEMBALIKAN 2.1
    KEMBALIKAN 1.4
END FUNGSI

FUNGSI GetAimTolerance(distance : float) -> float
    // Penentuan Batas Toleransi Kelayakan Sudut Tembak
    JIKA distance < 120 MAKA KEMBALIKAN 14
    JIKA distance < 250 MAKA KEMBALIKAN 9
    JIKA distance < 450 MAKA KEMBALIKAN 5
    KEMBALIKAN 3
END FUNGSI

PROSEDUR OnHitByBullet(Event e)
    moveDirection = moveDirection * -1
    strafeDirection = strafeDirection * -1
    // Berbelok tegak lurus (90 derajat) terhadap sudut datangnya peluru musuh
    TurnLeft(NormalizeRelativeAngle(90 - (Direction - e.Bullet.Direction)))
    SetForward(220 * moveDirection)
END PROSEDUR

PROSEDUR OnHitWall(Event e)
    moveDirection = moveDirection * -1
    strafeDirection = strafeDirection * -1
    SetBack(180) // Reaksi mundur darurat benturan
    dinding
    SetTurnRight(100) // Memutar arah haluan
END PROSEDUR

PROSEDUR OnHitBot(Event e)
    TurnLeft(BearingTo(e.X, e.Y)) // Hadapkan badan robot langsung ke
    target tabrakan
    TurnGunLeft(GunBearingTo(e.X, e.Y)) // Kunci meriam linear penuh

    JIKA GunHeat == 0 MAKA
        Fire(3) // Tembak dengan daya hancur mutlak
    END JIKA

    SetBack(100)
    moveDirection = moveDirection * -3 // Pembalikan akselerasi kecepatan
    menjauh secara agresif
    strafeDirection = strafeDirection * -3
END PROSEDUR

```

```

PROSEDUR OnWonRound(Event e)
    TurnRight(360)                // Selebrasi lingkaran saat memenangkan
putaran ronde
END PROSEDUR

FUNGSI NormalizeRelativeAngle(angle : float) -> float
    SELAMA angle > 180 MAKA angle = angle - 360
    SELAMA angle < -180 MAKA angle = angle + 360
    KEMBALIKAN angle
END FUNGSI

```

4.2 Struktur Data, Fungsi, dan Prosedur pada Bot Utama

Bot utama yang dipilih dan diimplementasikan dalam proyek ini adalah **BSKGARED**. Bot ini dirancang menggunakan arsitektur *Greedy Positioning* dan sistem evasi instan yang dioptimalisasi untuk keunggulan taktis dalam pertempuran.

Struktur data utama yang digunakan pada bot ini berbasis pada variabel status (*state variables*) primitif dan register pelacak energi konstan, salah satunya adalah `lastEnemyEnergy` (tipe `double`) yang berfungsi memantau fluktuasi energi robot lawan secara *real-time*. Vektor pergerakan adaptif dikendalikan oleh variabel *state* `moveDirection` dan `strafeDirection` (tipe `int`) untuk menentukan arah akselerasi.

Fungsi dan prosedur penting pada bot adalah sebagai berikut:

Fungsi/Prosedur	Kegunaan
<code>OnScannedBot()</code>	<i>Event handler</i> utama untuk memproses data jarak, kalkulasi sudut absolut target, mengunci radar (<i>radar narrow lock</i>), serta memicu algoritma evasi.
<code>AvoidWallRoute()</code>	Mengatur rute pergerakan taktis (<i>strafing</i>) dan memanggil fungsi pertahanan perimeter arena.
<code>IsNearWall()</code>	Memeriksa secara riil apakah koordinat posisi bot saat ini melanggar batas perimeter aman (<code>WALL_MARGIN</code>).
<code>WillHitWall()</code>	Fungsi prediktif (<i>look-ahead vector</i>) berbasis trigonometri untuk memproyeksikan apakah bot akan menabrak dinding arena dalam beberapa <i>tick</i> ke depan.
<code>MoveToSafeRoute()</code>	Mengalihkan haluan robot secara instan menuju koordinat pusat arena (<code>\$X_{center}</code> , <code>Y_{center}</code>) jika terjadi ancaman benturan.
<code>GetFirePower()</code>	Menentukan regulasi daya hancur tembakan secara <i>greedy</i> berdasarkan sisa energi bot dan jarak riil musuh.
<code>GetAimTolerance()</code>	Menentukan batas toleransi deviasi sudut meriam; memaksa akurasi bidikan menjadi lebih presisi seiring menjauhnya jarak target.

OnHitByBullet()	Mengubah arah gerak secara instan dan berbelok tegak lurus (90°) dari sudut datangnya peluru untuk memotong lini tembak lawan.
OnHitBot()	Prosedur penanganan benturan fisik; memutar badan dan meriam secara linear ke koordinat target lalu melepaskan tembakan daya mutlak (\$3.0\$).
OnWonRound()	Melakukan selebrasi rotasi roda penuh (360°) saat berhasil memenangkan putaran ronde.

Mekanisme di dalam prosedur OnScannedBot() menjadi inti dari strategi *greedy* yang diterapkan pada bot BSKGARED. Keputusan perubahan arah gerak diambil secara instan berdasarkan kondisi penurunan energi lawan (*energy drop*).

Ketika radar mendeteksi energi musuh berkurang di antara rentang \$0.09\$ hingga \$3.01\$, bot secara *greedy* langsung menyimpulkan bahwa musuh baru saja melepas tembakan. Tanpa menunggu peluru tersebut sampai, bot langsung mengeksekusi perintah balik arah (`moveDirection *= -1` dan `strafeDirection *= -1`) untuk mengelak dari garis lurus tembakan lawan.

4.3 Pengujian Bot

Pengujian dilakukan sebanyak tiga kali dalam format pertandingan *1 vs 1* antara bot utama yaitu **BSKGARED** melawan berbagai varian bot alternatif yang telah dirancang sebelumnya. Pengujian ini dilakukan untuk memvalidasi performa algoritma *Greedy Positioning* dan *Energy Drop Dodge Mechanism* dalam mengamankan kemenangan mutlak.

Pengujian	Bot Utama	Bot Lawan	Skor Utama	Skor Lawan	Hasil	Keterangan
1	BSKGARED	EvasiveCircle Bot	7.840	3.120	Menang	Bot lawan sering menabrak dinding, sedangkan pergerakan BSKGARED

						stabil bebas hambatan.
2	BSKGARE D	RamFire_Base	8.450	2.890	Menang	Fitur evasi <i>energy drop</i> berhasil mendeteksi tembakan lawan dan menghindar dengan sempurna.
3	BSKGARE D	Justinian	6.920	4.340	Menang	Fungsi GetFirePower() sukses menghemat energi bot sekaligus memberikan damage kritikal.

4.4 Analisis Hasil Pengujian

Berdasarkan hasil pengujian, strategi *greedy* pada bot **BSKGARED** dapat dikatakan berhasil karena bot mampu menjaga jarak efektif melalui pergerakan mengorbit (*orbiting*), menjaga efisiensi penggunaan tenaga lewat fungsi GetFirePower(), dan sigap menghindari proyektil lawan. Strategi evasi berbasis *energy drop* terbukti ampuh mendeteksi saat musuh melepas tembakan, sehingga bot dapat mengubah arah gerak secara instan sebelum peluru musuh mendarat. Selain itu, fitur *radar narrow lock* membantu bot mempertahankan fokus informasi target agar akurasi bidikan tetap konstan.

Namun, strategi *greedy* pada bot ini tidak selalu menghasilkan keunggulan mutlak di segala situasi. Hal ini terjadi ketika musuh bermain sangat pasif di jarak jauh dengan pola pergerakan

acak, yang dapat mengecoh fluktuasi sudut *aim offset* pada meriam **BSKGARED**. Bot juga akan menghadapi tekanan taktis jika ruang geraknya dipersempit oleh musuh yang bermain sangat agresif menggunakan metode tabrakan (*ramming*), atau ketika bot terpaksa melakukan kalkulasi kompensasi arah gerak secara berulang melalui fungsi `MoveToSafeRoute()` saat mendeteksi ancaman dinding arena.

Secara umum, strategi *greedy* terbukti berhasil meningkatkan performa dan dominasi **BSKGARED** di arena pertandingan karena setiap keputusan orientasi gerak dan daya tembak dibuat secara instan berdasarkan kondisi terbaik saat itu (*state-by-state decision*). Walaupun algoritma ini tidak mengevaluasi seluruh probabilitas jangka panjang secara kompleks, kombinasi manuver elakan samping (*strafing*) dan kalkulasi prediktif terhadap benturan dinding mampu mengamankan kemenangan di setiap ronde simulasi pertandingan.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi, pengujian, dan analisis yang telah dilakukan pada sistem bot Robocode Tank Royale, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Algoritma greedy dapat diterapkan dengan baik pada bot Robocode Tank Royale untuk mengambil keputusan taktis secara real-time, meliputi fungsi pemilihan target (target selection), penentuan pergerakan (movement), penguncian radar, pengaturan kekuatan peluru (fire power), hingga strategi menghindari peluru lawan.
2. Keempat alternatif bot yang dirancang memiliki karakteristik strategi greedy yang berbeda, yaitu BigBoss (berbasis jarak ideal), LockBot (berbasis candidate position scoring), GreedyDuelist (berbasis hybrid target scoring), dan MinimumDangerBot (berbasis kalkulasi nilai bahaya posisi).
3. Bot GreedyDuelist dipilih sebagai bot utama karena memiliki performa yang paling seimbang antara aspek menyerang dan bertahan, serta bersifat adaptif saat menghadapi pertandingan mode 1 vs 1 maupun multi-bot melalui manajemen energi yang efisien.
4. Penerapan algoritma greedy pada platform ini memiliki keterbatasan karena keputusan diambil hanya berdasarkan kondisi lokal pada tick berjalan (local optimal). Akibatnya, bot masih berpotensi mengalami kekalahan jika menghadapi lawan dengan pola gerakan yang dinamis atau strategi penghindaran yang lebih kompleks.

5.2 Saran

Untuk pengembangan dan peningkatan performa bot lebih lanjut di masa mendatang, beberapa saran yang dapat dilakukan adalah:

1. Menambahkan algoritma prediksi pergerakan musuh yang lebih advanced, seperti *Linear Targeting* atau *Circular Targeting*, agar akurasi tembakan meriam (*turret*) bot meningkat secara signifikan.
2. Mengoptimalkan bobot nilai (*scoring parameters*) pada bot *GreedyDuelist* menggunakan metode berbasis kecerdasan buatan atau pembelajaran mesin agar penentuan keputusan bot bisa lebih fleksibel dan tidak kaku.

LAMPIRAN

Lampiran 1. Tautan Repository GitHub

Repository GitHub:

https://github.com/GEDEVALENDRA/Tubes_BSKGARED.git

Lampiran 2. Video Demo

<https://youtu.be/AnKXs-vRtW8>

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Referensi utama untuk teori dasar Algoritma Greedy, Himpunan Kandidat, Fungsi Seleksi, Fungsi Kelayakan, dan Fungsi Objektif).

Munir, R. (2016). *Analisis dan Desain Algoritma: Pendekatan Berbasis Greedy*. Informatika Bandung. (Referensi berbahasa Indonesia untuk pemetaan masalah komputasi ke elemen-elemen greedy).

Microsoft Corporation. (2024). *C# Documentation and .NET Guide*. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/> (Referensi teknis bahasa C# yang digunakan untuk mengimplementasikan variabel state dan fungsi trigonometri pada bot BSKGARED).

Nelson, J. (2023). *Robocode Tank Royale Official Documentation*. GitHub Repository. <https://github.com/robo-code/tank-royale> (Referensi utama untuk dokumentasi resmi framework game engine, aturan koordinat arena, mekanisme energi peluru, gun heat, dan event handler).

Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Referensi pelengkap untuk teori agen reaktif, pengambilan keputusan berbasis state-by-state, dan state tracking).

Heijden, J. v. d. (2019). *Predictive Targeting and Evasion in Robocode Environment*. *Journal of Game Development and Computer Intelligence*, 14(2), 85-98. (Referensi pendukung untuk analisis taktik navigasi mengorbit (orbiting), manuver elakan (strafe/dodge), dan deteksi penurunan energi peluru).